HW 8

Dylan Thornburg
[SCI 163] CSEN 179

6.4

Subproblems: Scan for words and factor in previous outcomes

Input: string s[1...n]
Output: P[0...n] = false for all points out the start

```
Sol(s):
P[0] = true //base
for (i = 1...n+1):
  for (j = 1...i+1):
    if dict(s[j-1 to i]) = true
    and P[j-1] = true
      P[i] = true
      break
return P[n]
```

turns out the best

0 = false
1 = true

	i=1	2 (it)	3	4	5 (was)	6	7	8	9	10	11	12
j=1	0	1	0	0	0							
2	x	0	0	0	0							
3	x	x	0	0	1							
4	x		x	0	0							
5	x			x	0							
6	x				x							
7	x											
8	x											
9	x											
10	x											
11	x											
12	x											1

← if there is 1 in col 12, we will return true b/c there is a sentence

This algo will update the P[] array so any entry where there's a true will mark the end of a word. If there is a coherent sentence, P[n] will be true. To figure out the corresponding words, loop through P[] and s[] and save the letters from s[]. Any time P[] = true, save that word then start fresh.

6.7

Subproblems: find max palindrome for a sequence. Can be split up and use past results like so:

A → AC → ACG → ACGT

A	C	G	T
	C	G	T
		G	T
			T

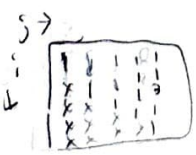
```
Sol(x[], n):
init L[n][n]
for (i = 1...n):
    L[i][i] = 1 // single letters are 1
for l = 2...n:
  for i = 1...n-l+1:
    j = i+l-1
    if x[i] = x[j]:
      if l == 2: L[i][j] = 2
      else: L[i][j] = 2 + L[i+1][j-1]
    else: L[i][j] = max{L[i+1][j], L[i][j-1]}
return L[1][n]
```

$L[i,j] = \begin{cases} i=j: 1 \\ x[i]=x[j]: L[i+1,j-1]+2 \\ \text{else: } \max\{L[i+1,j], L[i,j-1]\} \end{cases}$

x[i] matches x[j] so we can have both → L[i+1,j-1]+2

can only have one → max{L[i+1,j], L[i,j-1]}

ACGTG
CGTG
GTG
3
TG
G
1



6.8

```
LCSString(x[], y[], n, m):
L[m+1][n+1]
Max = 0
for i = 0...n:
  for j = 0...m:
    if (i or j = 0): L[i][j] = 0 // set all bases to 0

for (i = 1...n):
  for (j = 1...m):
    if (x[i-1] = y[j-1]):
      L[i][j] = L[i-1][j-1] + 1
      Max = max{Max, L[i][j]}
    else:
      L[i][j] = 0
return Max
```

Subproblems: Need to determine when characters match consecutively. Can do this by comparing every character and remembering previous results when determining value.

example
x[] = abcne
y[] = zabcyw

i \ j →	0	1	2	3	4	5	6	7
0	0	0	0	0	0	0	0	0
1	0	0	1	0	0	0	0	0
2	0	0	0	2	0	0	0	0
3	0	0	0	0	3	0	0	0
4	0	0	0	0	0	0	0	0
5	0	0	0	0	0	0	0	0
6	0	0	0	0	0	0	0	0

whatever the highest val in the 2D array is, is the answer.

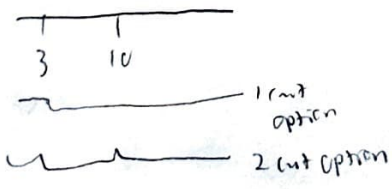
prev result w/ contiguosness

$$L(i,j) = \begin{cases} \text{if } x_{i-1} = y_{j-1}: L(i-1, j-1) + 1 \\ \text{else: } 0 \end{cases}$$

6.9

```
minMultCost(m[], n):
```

Subproblems:



Recursive relation

I know this problem is suppose to be similar to chain mmult, but I just cannot figure out exactly what is being asked. I will just take the logs.

6.11 This problem is similar to 6.8. Now that we're looking for subsequences, the matrix will factor in previous results more and the answer will be found at array index L[m][n].

Subproblems: Now we will store the results even when the strings stop matching contiguously. This will result in the bottom right (m,n) index storing the final result.

```
LCSSequence(x[],y[],n,m):
L[m+1][n+1]
for i=0...n:
    for j=0...m:
        if(i or j=0): L[i][j]=0
for(i=1...n):
    for(j=1...m):
        if(x[i-1]=y[j-1]):
            L[i][j]=L[i-1][j-1]+1
        else:
        L[i][j]=max{L[i-1][j], L[i][j-1]}
return L[m][n]
```

x: abcne
y: bcane

i ↓ \ j →	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	0	1	1	1
2	0	1	1	1	1	1
3	0	1	2	2	2	2
4	0	1	2	2	3	3
5	0	1	2	2	3	4 ✓

$$L(i,j) = \begin{cases} x_{i-1}=y_{j-1}: L(i-1,j-1)+1 \\ \text{else: } \max\{L(i-1,j), L(i,j-1)\} \end{cases}$$

6.22

```
SubsetSum(a[],n,t):
create matrix S[0,..n][0,..t]
for i=0...n:
    S[i][0]=true
for i=1...t:
    S[0][i]=false
for i=1...n:
    for j=1...t:
        if(a[i]≤j):
            if(S[i-1][j-a[i]]=true):
                S[i][j]=S[i-1][j-a[i]]
            else:
                S[i][j]=S[i-1][j]
        else:
            S[i][j]=S[i-1][j]
return S[n][t]
```

↑ returns true if possible

1,5,11,5 → 10

1 is true
0 is false

	0	1	2	3	4	5	6	7	8	9	10
	0	1	0	0	0	0	0	0	0	0	0
1 →	1	1	1	0	0	0	0	0	0	0	0
5 →	2	1	1	0	0	0	1	1	0	0	0
11 →	3	1	1	0	0	0	1	1	0	0	0
5 →	4	1	1	0	0	0	1	1	0	0	1

if S[n][t]=1 then true

$$S(i,j) = \begin{cases} \text{if } a_i < j: \\ S(i,j) = S(i-1, j-a_i) \\ \text{else:} \\ S(i,j) = S(i-1,j) \end{cases}$$

← this was unnecessary but I'm not erasing

6.26

```
highestScore(x[], y[], n, m, γ):
  create matrix S[0...n][0...m]
  for (i=0...n):
      S[i][0]=0
  for (i=0...m):
      S[0][i]=0

  for i=1...n:
    for j=1...m:
      S[i][j]= max{ (S[i-1][j]),
                    (S[i][j-1]),
                    (S[i-1][j-1])}
                  + γ(x[i], y[j]))

  return S[n][m]
```

	0	1	2	3	4	5	6
	T	A	-	C	G	C	A
0	+	γ(-,T)	γ(-,A) γ(-,T)	...			
1	A	γ(-,T)+ γ(A,T)	← max{ γ(-,T), γ(-,A)+γ(-,T), γ(-,T)+γ(A,T) or (i-1,j), (i,j-1), (i-1,j-1) } + (i,j)				
2	T	⋮					
3	-	⋮					
4	G						
5	C						
6	C						

$$S(i,j)=\max\{\gamma(S(i-1,j)),\ \gamma(S(i,j-1)),\ \gamma(S(i-1,j-1))\}+\gamma(x[i],y[j])$$

Subproblems:

Need to find max score from two lists. Use a 2D array to keep track of past scores